



Universidad
Indoamérica

UNITÉ 5 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Tubes et composition de processus

Universidad Tecnológica Indoamérica

Laboratoire TD4

Comment deux processus isolés communiquent-ils?

Flux d'octets

Tubes (cette unité), FIFO (unité 3), sockets (unité 8)

Notifications

Signaux (unité 6)

Mémoire partagée

mmap et mémoire partagée (unité 6), threads (unité 7)

Le tube est le plus simple et le plus utilisé: chaque `|` du shell en est un.

Un tube: pipe(fds)

- Crée un tampon dans le noyau et deux descripteurs: `fds[0]` pour lire et `fds[1]` pour écrire
- Un seul sens, un flux d'octets sans messages et sans nom
- Seuls les processus qui héritent des descripteurs le partagent: **on le crée avant fork**
- Chaque processus **ferme l'extrémité qu'il n'utilise pas**

C tds/td4/mypipe.c

```
exit_if(pipe(fds) == -1, "pipe");
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    exit_if(close(fds[1]) == -1, "close"); /* ch. */
    char buffer[READ_MAX];
    ssize_t n = read(fds[0], buffer, READ_MAX); /* ne */
    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n)
    fprintf(stderr, "I am the child, I read %zd bytes an
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
```

Les règles d'un tube

Lire avec le tube vide	bloque... ou renvoie 0 s'il ne reste aucun écrivain
Écrire avec le tube plein	bloque jusqu'à ce que quelqu'un lise
Écrire sans aucun lecteur	signal SIGPIPE (ou EPIPE s'il est ignoré)
Écrire $\leq$ PIPE_BUF (4096 o)	atomique: ne se mélange pas avec les autres

Capacité par défaut sur la machine du cours:

Sortie réelle

```
$ python3 -c 'import os, fcntl; r, w = os.pipe();  
print(fcntl.fcntl(w, fcntl.F_GETPIPE_SZ), "bytes")'  
65536 bytes
```

10, 300, 10 000 messages: que se passe-t-il?

chronologie (modèle, µs)

parent



enfant

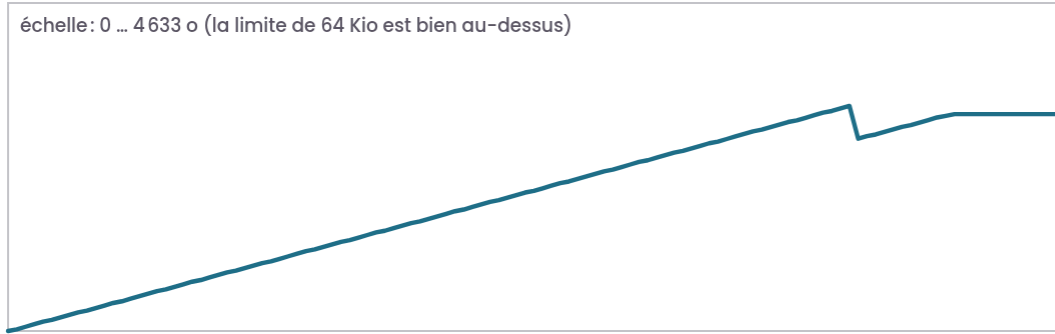
fork

read -> 500 B

exit

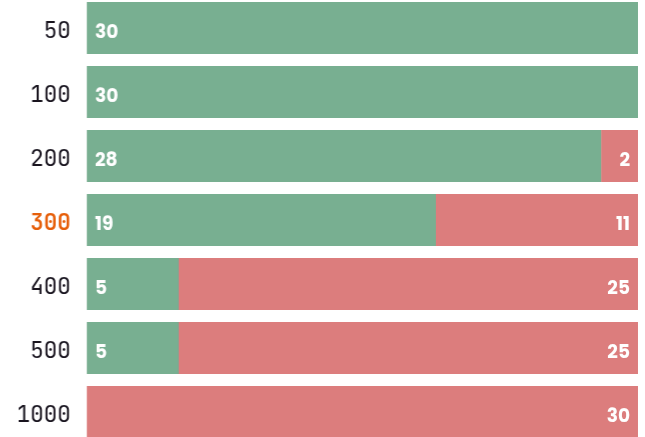
octets dans le tube

échelle: 0 ... 4 633 o (la limite de 64 Kio est bien au-dessus)



166 µs

mesure réelle: 30 exécutions par taille



vert: statut 0 · rouge: 141 (SIGPIPE)
messages

Fermer ou non `fds[0]` dans le parent

Sortie réelle

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"  
I am the child, I read 500 bytes and I leave  
exit status of mypipe: 141
```

Sortie réelle

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by timeout: it was blocked)"  
I am the child, I read 500 bytes and I leave  
exit status: 124 (124 = killed by timeout: it was blocked)
```

Avec l'extrémité fermée, le parent reçoit SIGPIPE (141 = 128 + 13) quand l'enfant part. Sans la fermer, le parent est son propre « lecteur » : il remplit les 64 Kio et bloque pour toujours (124 = tué par `timeout`). Lancé avec `&`, le shell indique le statut de sortie.

Brancher le programme d'un autre: `dup2` + `exec`

C tds/td4/upper.c

```
/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");          /* 1 already closed */
execlp("ps", "ps", (char *)NULL);              /* "ps" to be executed */
exit_if(1, "execlp ps");
```

Sortie réelle

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?            00:00:25 UVICORN
 9771 ?            00:00:03 SYSTEMD
 9801 ?            00:00:00 (SD-PAM)
```

`ps` ne sait rien des tubes: il écrit sur son descripteur 1, que `dup2` a relié au tube. L'enfant lit jusqu'à la fin de fichier et convertit en majuscules.

Ce que fait le shell avec

```
ls -l | tr -d '[:blank:]'
```

1. `pipe(fds)`
2. `fork` → enfant 1: `dup2(fds[1], 1)`, ferme les deux, `exec("ls")`
3. `fork` → enfant 2: `dup2(fds[0], 0)`, ferme les deux, `exec("tr")`
4. Le shell ferme ses deux extrémités et attend les deux enfants

Les tables de descripteurs, pas à pas

pipeline

pipeline

0	terminal
1	terminal
2	terminal
3	
4	
5	
6	

enfant 1

ls -l demo

0	terminal
1	tube 0 · écriture
2	terminal
3	
4	
5	
6	

enfant 2

grep txt

0	tube 0 · lecture
1	tube 1 · écriture
2	terminal
3	
4	
5	
6	

enfant 3

wc -l

0	tube 1 · lecture
1	terminal
2	terminal
3	
4	
5	
6	

tube 0 · écrivains 1 · lecteurs 1

tube 1 · écrivains 1 · lecteurs 1

ligne 60: close(4)

N commandes et le statut du pipeline

C tds/td4/pipeline_args.c

```
if (pid == 0) {
    if (in != STDIN_FILENO) {
        exit_if(dup2(in, STDIN_FILENO) == -1, "dup2");
        exit_if(close(in) == -1, "close");
    }
    if (fds[1] != -1) {
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1,
            exit_if(close(fds[0]) == -1 || close(fds[1])
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
}
```

Sortie réelle

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortie réelle

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Sortie réelle

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Comme le shell, il renvoie le statut de la **dernière** commande (`set -o pipefail` change cette règle dans bash).

Au-delà de `pipe`

`pipe2`

`O_CLOEXEC` empêche le tube de fuir vers les programmes lancés avec `exec` ;
`O_NONBLOCK` pour ne pas bloquer.

`splice`

Déplace des données entre un tube et un fichier ou un socket sans les copier dans le processus.

Capacité

`fcntl(F_SETPIPE_SZ)`
l'agrandit jusqu'à
`/proc/sys/fs/pipe-max-size`.

TD4 corrigé + défi IA

- `mypipe` avec 10, 300 et 10 000 messages; avec et sans le bogue
- `upper` : `ps` en majuscules
- `pipeline_args ls -l -- grep txt -- wc -l`

Bash

```
make -C tds && tds/tests/run.sh td4
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Quand un `read` sur un tube renvoie-t-il 0?
2. Pourquoi `mypipe 10000` se termine-t-il avec 141 et `mypipe_bug` ne se termine-t-il jamais?
3. Dans `a | b | c`, combien d'appels à `pipe`, `fork` et `close` le shell fait-il?